scientific reports

Check for updates

OPEN

Deep multi-metrics learning for mobile app defect prediction using code and process metrics

Ahmed Abdu[1], Hakim A. Abdo[6], Inam Ullah[2], Jawad Khan[3], Yeong Hyeon Gu[4]✉ & Redhwan Algabri[5]✉

The rise in mobile apps necessitates precise defect prediction to aid developers in resource allocation. The efficacy of defect prediction relies on data representation and model selection. However, existing research relies on isolated data sources, limiting models' ability to capture the complex interplay of code and process metrics in app development. This paper addresses this limitation by proposing a Deep Multi-Metrics Learning Model (DMLM), which leverages code metrics from the current code version and process metrics from previous releases. A deep convolutional neural network (CNN) is employed to capture intricate patterns within these metrics, enabling more accurate predictions. Experimental evaluations using nine real-world Android apps from Git Android repositories demonstrate that DMLM outperforms state-of-the-art approaches under non-effort-aware conditions regarding Area Under the Curve (AUC), F1 scores, and Matthews correlation coefficient (MCC). The experimental evaluation also demonstrates that the DMLM model outperforms existing baseline methods in PofB20 under effort-aware conditions. The findings highlight the efficacy of DMLM in enhancing mobile app defect prediction performance across both non-effort-aware and effort-aware contexts.

Keywords Mobile app defect prediction, Code metrics, Process metrics, Deep neural network

The proliferation of mobile devices has fueled an unprecedented expansion of mobile applications, particularly on the Android platform. This growth is accompanied by increasing user expectations for innovative features, seamless functionality, and robust performance[1–4]. As developers strive to meet these demands, the frequent release cycles required to deliver updates and enhancements introduce significant challenges. Among these, the inadvertent introduction of defects during development and maintenance processes represents a critical issue, as defects can degrade user experience, compromise security, and harm the reputation of applications. Addressing this issue requires effective strategies to predict and mitigate defects during the development lifecycle, ensuring the release of high-quality, reliable mobile applications[5,6].

Traditionally, defect prediction efforts in mobile applications have focused on utilizing static code metrics[7–9]. These metrics, derived from the structural properties of the codebase, provide valuable insights into the internal characteristics of software components. However, code metrics often fail to capture mobile application development's dynamic and evolutionary nature, where frequent updates, feature additions, and modifications significantly impact software quality.

In response to these limitations, researchers have explored the importance of process metrics that track changes and modifications across software versions[10–15]. Metrics such as lines of code added or deleted, code churn, and prior defect density offer a temporal dimension that reflects App development's iterative and evolutionary aspects. While these metrics provide complementary insights, their application in isolation has notable drawbacks. For instance, increased added lines of code may not necessarily signal defect-prone areas if the changes involve well-designed and thoroughly reviewed features. Conversely, static metrics may overlook critical evolutionary factors, such as patterns of defects emerging from frequent updates or areas affected by complex code changes.

Despite the substantial progress made by employing either code metrics or process metrics in isolation, significant limitations remain that hinder the accurate and reliable prediction of defects in mobile Apps. Code

[1]School of Information, Xi'an University of Finance and Economics, Xi'an 710100, China. [2]Department of Computer Engineering, Gachon University, Seongnam 13120, Republic of Korea. [3]School of Computing, Gachon University, Seongnam 13120, Republic of Korea. [4]Department of Artificial Intelligence and Data Science, College of Software and Convergence Technology, Sejong University, Seoul 05006, Republic of Korea. [5]Department of Computer Science and Engineering, Sejong University, Seoul 05006, Republic of Korea. [6] Department of Computer Science, Hodeidah University, Al-Hudaydah, Yemen. ✉email: yhgu@sejong.ac.kr; redhwan@sejong.ac.kr

metrics, while effective in quantifying structural complexity and syntactic characteristics, do not capture the dynamic and iterative nature of mobile app development. Conversely, process metrics provide valuable insights into historical modifications and developer activity but lack the capacity to represent inherent design and structural flaws in the current codebase. These shortcomings highlight a pressing need for innovative methodologies capable of integrating both perspectives to model the multifaceted nature of defect generation in mobile systems.

In response to this challenge, the present study introduces a new approach that systematically leverages the complementary strengths of static code metrics and process metrics within a unified, multi-metrics framework. Code metrics are employed to assess the structural integrity, complexity, and maintainability of the current release. In contrast, process metrics capture the evolutionary patterns, code churn, and historical defect-proneness across previous versions. The integration of these heterogeneous feature spaces provides a more holistic representation of software quality, enabling the identification of modules that are not only structurally complex but also evolutionarily unstable. The proposed method utilizes a sophisticated deep CNN architecture designed to uncover intricate patterns and interactions between these various types of metrics. This innovative framework enhances predictive accuracy and provides actionable insights that support informed decision-making during mobile app development.

Unlike conventional mobile App defect prediction methods, DMLM is specifically designed for practical, real-world applications. By directly extracting code and process metrics from Git repositories, DMLM eliminates extensive manual feature engineering and adapts easily to continuous integration pipelines. This scalability enables DMLM to handle evolving codebases and large datasets, providing actionable insights to developers with minimal overhead. Consequently, DMLM not only improves predictive performance but also supports seamless integration into modern software development workflows. Specifically, this paper makes the following key contributions:

1. We propose a new approach, named the DMLM to improve mobile app defect prediction by leveraging code metrics extracted from the current code version with process metrics related to the code changes of the previous releases. This approach addresses the limitations of using process change metrics alone by integrating the strengths of code metrics, which capture inherent design complexities, and process change metrics, which reflect dynamic evolution patterns.
2. We construct deep CNN to harness the benefits of these metrics and capture the interactions between code metrics and process metrics, leading to enhanced performance of mobile app defect prediction.
3. Several experiments are conducted on large-scale real-world Android projects to evaluate the performance of DMLM. The results demonstrate that the fusion of code metrics with process metrics significantly enhances the effectiveness of mobile app defect prediction.

The remainder of this paper is structured as follows: Sect. “Related work” covers related work. The motivation underlying this work is presented in Sect. “Motivation example”. Section "The proposed methodology" illustrates the proposed approach for metric extraction, model construction, and defect prediction. Section “Experimental setup” describes the experimental setup, while Sect. "Results analysis and discussion" analyzes and discusses the results. Sect. "Threats to validity" addresses threats to validity, and Sect. “Conclusion” concludes the study.

Related work

Defect prediction is a topic of significant interest in software engineering, aiming to identify defect-prone areas in software projects to enhance software quality and maintainability[16–19]. Mobile apps run on various mobile devices, such as tablets and smartphones, have become integral to modern technology and have transformed how users interact with software products and services. This increasing trend in mobile apps has resulted in challenges and opportunities for software defect prediction, making it crucial to develop effective methodologies to ensure the quality, reliability, and security of mobile apps in the software development lifecycle.

Several studies have focused on developing effective approaches to identify and predict defects in mobile apps. Early studies mainly used traditional code metrics like lines of code, cyclomatic complexity, etc., to develop defect prediction models for mobile apps. Dong et al[7]. presented a defect prediction approach using deep neural networks based on Android APK files. They conducted an explorative study to generate smali2vec, a new approach for generating representations of Android APK files. Their research addressed the need for accurate defect prediction in the context of Android app development, where the implications of faults are substantial. Their study showcased the potential of deep neural networks for enhancing defect prediction in mobile apps, providing valuable insights for further advancements in the field. Malhotra[8]introduced a framework to detect defective classes by leveraging object-oriented metrics. They conducted experiments on seven popular Android apps, revealing varying performance outcomes across 18 classification models. Ricky et al[9]. presented an SVM-based approach for app defect prediction. Through experiments on five datasets, they demonstrated that their support vector machines (SVM) method outperformed decision trees regarding predictive performance.

Due to frequent updates in mobile apps, researchers recently used process metrics, such as the number of commits and the frequency of code changes for mobile app defect prediction, to identify patterns indicating defect-prone areas. Catolino et al.[10]examined the significance of process metrics, analyzed how four traditional classifiers performed, and assessed the effectiveness of ensemble learning methods in the context of JIT Android app defect prediction. Their research encompassed 14 apps, involving a total of 42,543 commits. The findings of their study indicated that Naive Bayes (NB) outperformed other classifiers, including some ensemble learning approaches. Zhao et al[11]. suggested a novel approach using Simplified Deep Forest (SDF) for conducting JIT Android app defect prediction. The SDF model modifies the traditional deep forest, removing the multi-grained scanning operation developed for high-dimensional metrics spaces. They evaluated their approach

on 10 Android apps, demonstrating the promising direction for advancing JIT mobile app defect prediction. Cheng et al[12]. proposed an approach based on kernel adversarial learning (KAL) for addressing the cross-project JIT mobile app defect prediction. The KAL method incorporates two crucial components: Firstly, it employs a kernel-based analysis technique to transform the commit into a metrics space with higher dimensions, facilitating the extraction of representative features. Secondly, the approach utilizes adversarial learning to derive feature embeddings for model construction. Huang et al[13]. proposed a multi-task learning (MTL-DNN) approach to address the JIT mobile app defect prediction challenge. To assess the efficiency of their method, the authors evaluated the MTL-DNN method on 15 Android apps. Jiang et al[20]. introduced COTL, a cross-project framework for mobile app defect prediction. It operates in two phases: offline and online. During the offline phase, a cross-domain structure-preserving projection algorithm minimizes distributional disparities between projects. In the online phase, target data arrives incrementally over time.

In the mobile app defect prediction aspect, code metrics are limited in capturing mobile app development's dynamic and evolutionary elements that emerge from changes across versions. While effective at evaluating code's structural complexity and quality, these metrics do not account for the iterative modifications prompted by new user requirements or frequent updates. Conversely, approaches relying exclusively on code change metrics face their challenges. While metrics like code churn effectively reflect developer activities and software evolution, they fail to consider intrinsic code complexity. For example, a highly intricate module with minimal churn may still be prone to defects due to its complexity. Furthermore, change metrics only apply to files that have undergone modifications, reducing their scope of analysis.

To address the abovementioned limitations, this study proposes a novel methodology called DMLM, incorporating code metrics from the current code version and process metrics that capture historical code changes. Code metrics provide insights into the structural and qualitative aspects of the codebase, while process metrics reflect the application's evolutionary behavior. This approach uses a deep CNN framework to model complex interactions between these distinct metric types. It offers a more holistic and accurate prediction of defect-prone areas in mobile applications. This innovative strategy significantly enhances the predictive capability of mobile app defect models, advancing the understanding of defect prediction in the context of evolving software systems.

Motivation example

The rapid proliferation of mobile applications has made software quality assurance a critical priority. End users expect apps that are reliable, fast, and secure. However, frequent updates, evolving features, and diverse device environments often lead to defect introduction. This work is motivated by the practical challenge faced by mobile app developers who must deliver updates quickly while minimizing user-reported crashes. In real projects, ignoring either metric type leads to inefficient bug detection, wasted effort, and increased maintenance costs. To illustrate the motivation behind this proposed work, let's consider a practical example depicted in Fig. 1, which describes a part code of the Android Gallery application. This code enables users to capture, store, and organize images. A crucial part of this functionality is the 'storeImages()' function, which saves captured images into the device's repository for later retrieval. In the previous release of this App, the storeimages() method lacked the while (!uriTask.isComplete()); loop. The code called uriTask.getResult() immediately after askSnapshot.getStorage().getDownloadUrl(), which often returned an IllegalStateException because it tried to fetch the download URL before the upload task was finished. This caused a race condition defect. After that, in the next release, developers identified the race condition. Their intent was correct: ensure the upload is complete before fetching the URL. Their solution was to add the line while (!uriTask.isComplete()); to create a blocking wait. This solved the race condition.

Existing mobile App defect prediction approaches rely primarily on process metrics or static code metrics in isolation, each with apparent drawbacks:

- A model that relies solely on process metrics would primarily capture the historical record of modifications, such as the fact that the file was recently altered. It would observe a relatively small number of lines added (LA), in this case, the insertion of a busy-wait loop. However, without access to the semantic meaning of the added lines, the model is incapable of assessing the quality of the change. Such a modification could represent either an effective bug fix or the introduction of a critical defect. In general, process metrics (e.g., code churn, lines added or deleted) reflect the temporal evolution of software artifacts but fail to capture underlying design flaws. Consequently, a module exhibiting limited churn yet possessing high structural complexity may still be highly prone to defects–an aspect that process metrics alone are insufficient to reveal.
- Static code metrics (e.g., Lines of Code (LOC), cyclomatic complexity) provide valuable insights into the structural and syntactic characteristics of software; however, they fail to capture the evolutionary changes that occur across successive versions. Such metrics are inherently limited in modeling the dynamic development practices typical of mobile applications, where frequent updates and continuous feature modifications are prevalent. A predictive model that relies solely on static code metrics of the current "fixed" version would merely recognize a method as syntactically valid. Although the metrics might reflect minor increases in LOC or slight variations in complexity, these changes would be insufficient to indicate a high defect risk in isolation. Crucially, the underlying defect, such as blocking the main thread, represents a runtime, architectural vulnerability that remains undetectable through static analysis alone and would therefore likely be misclassified as low risk.

As mentioned above, current approaches to mobile application defect prediction predominantly rely on either process metrics or static code metrics in isolation, which limits their effectiveness. By focusing on a single type of metric, these methods often fail to generalize across diverse projects and fail to capture the multifaceted nature

```
public class SortGalary extends AppCompatActivity implements AdapterView.OnItemSelectedListener
{
   // Declare variables here
    @Override
    protected void onCreate(Bundle savedInstanceState)
    {
        // Assign values here
        // Link database here
        // called this function retrieve infornation
        // called for spinner.setOnItemSelectedListener
        // Code for ActivityResultLauncher<Intent> a
        // Code for uploadImage.setOnClickListener
        saveButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                storeimages();
            }
        });
    }
    public void storeimages()
    {
        StorageReference storageReference =
           FirebaseStorage.getInstance().getReference().child("Images")
                .child(uri.getLastPathSegment());
        AlertDialog.Builder builder = new AlertDialog.Builder(Add.this);
        builder.setCancelable(false);
        builder.setView(R.layout.progress_layout);
        AlertDialog dialog = builder.create();
        dialog.show();
        storageReference.putFile(uri).addOnSuccessListener(
                    new OnSuccessListener<UploadTask.TaskSnapshot>() {
            @Override
            public void onSuccess(UploadTask.TaskSnapshot taskSnapshot) {
                Task<Uri> uriTask = taskSnapshot.getStorage().getDownloadUrl();
                while (!uriTask.isComplete());
                Uri urlImage = uriTask.getResult();
                imageURL = urlImage.toString();
                uploadData();   // call this function to upload all data
                dialog.dismiss();
            }
        }).addOnFailureListener(new OnFailureListener() {
            @Override
            public void onFailure(@NonNull Exception e) {
                dialog.dismiss();
            }
        });
    }
}
```

Fig. 1. Mobile App code as motivation example.

of software quality. Moreover, they face significant challenges in effort-aware scenarios, where development resources are constrained, and it is crucial to prioritize defect-prone files to maximize the efficiency of testing and maintenance activities.

This gap between evolutionary behavior (process metrics) and structural quality (code metrics) creates the need for an integrated and adaptive defect prediction framework. The proposed DMLM model surmounts these limitations through a fused learning paradigm that concurrently integrates both code and process metrics. This approach enables a context-aware prediction by leveraging static code metrics to evaluate the structural complexity of the current code version. In contrast, process metrics provide the evolutionary context of changes from previous releases. A deep CNN is then employed to model the intricate, non-linear interactions between these disparate metric types. This synthesis allows DMLM to identify high-risk patterns that are invisible to single-metric approaches, such as a specific, hazardous code change made to a file with a particular evolutionary history, leading to a more holistic and accurate assessment of defect proneness. Consider a method like storeImages() in the Android Gallery app shown in the above example, where a busy-wait loop was added to prevent a race condition. While process metrics alone would show the change in code churn, they would not indicate the complexity of this change or whether it was beneficial or harmful. Static code metrics might show that the change didn't significantly affect the overall cyclomatic complexity, yet this wouldn't capture the potential runtime defect it could cause. DMLM highlights this issue by combining both sets of metrics and utilizing deep learning, recognizing that the specific code change, combined with the file's previous development history, could introduce a defect, such as blocking the main thread or causing performance degradation.

Furthermore, DMLM is adaptive, making it suitable for dynamic, evolving mobile applications. It doesn't treat every change in isolation but considers the broader context, ensuring that defect prediction is informed by both the static state of the code and its evolutionary journey. This results in a more accurate and context-aware

prediction model, which is particularly valuable in effort-aware scenarios where limited resources necessitate prioritizing defect-prone areas of the codebase. By bridging the gap between static analysis and process-driven evolution, DMLM offers a comprehensive solution that significantly improves defect prediction accuracy, especially in the face of frequent updates and evolving codebases typical in mobile app development.

The proposed methodology

This section presents an outline of our proposed methodology. Our approach involves two primary stages: 1) data collection and feature extraction, and 2) learning model and defect prediction, as illustrated in Fig. 2 and Fig. 3. Below is a detailed explanation of each step:

Data collection and feature extraction

Data collection

To construct a comprehensive dataset for our research, we selected nine Android application packages: Bluetooth, Browser, Camera, Contact, DocumentsUI, Gallery2, Email, Messaging, and TV App. These apps were selected because they collectively represent a wide range of functionalities commonly used in real-world mobile environments, ensuring the diversity and generalizability of this study. For example, Messaging and Bluetooth are communication-intensive modules that undergo frequent updates to accommodate evolving protocols and user requirements. Camera and Gallery are essential multimedia applications that require significant user interaction, real-time processing, and device integration, making them susceptible to defects from performance or resource management issues. Browser and Email represent applications with high interaction complexity, where security, data handling, and continuous updates are critical. Similarly, the TV app encompasses a category of applications focused on entertainment, featuring streaming and rendering capabilities. Extensive codebases and elevated defect rates often characterize these applications. Finally, Contacts and Documents UI highlight utility-oriented applications that manage personal data and file systems, where reliability and correctness are paramount.

To obtain the source code of these application packages, we access the Git repository from Google (https://android.googlesource.com). This repository serves as a central hub for the Android open-source project, providing access to the source code of various Android components and applications. Multiple releases of each application package were collected to capture their evolution over time. Using multiple releases allows developers and testers to observe potential changes in defect-prone areas across different versions of the applications. By analyzing multiple releases, we aim to enhance the robustness and generalizability of our defect prediction models. Table 1 shows an overview of the characteristics of the nine Android app packages across various releases, including the application name, the Android version, the total classes, the total LOC, and the average defective rate.

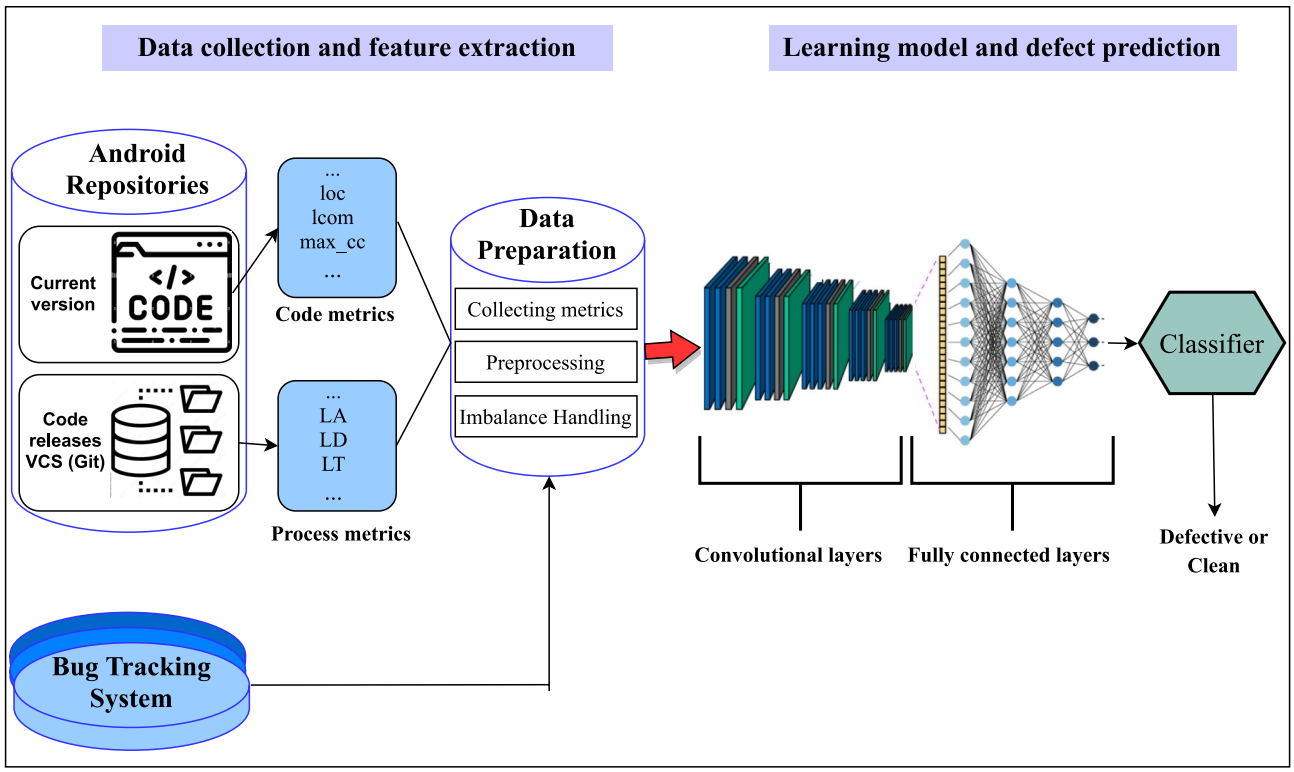

Fig. 2. Overview of the proposed DMLM.

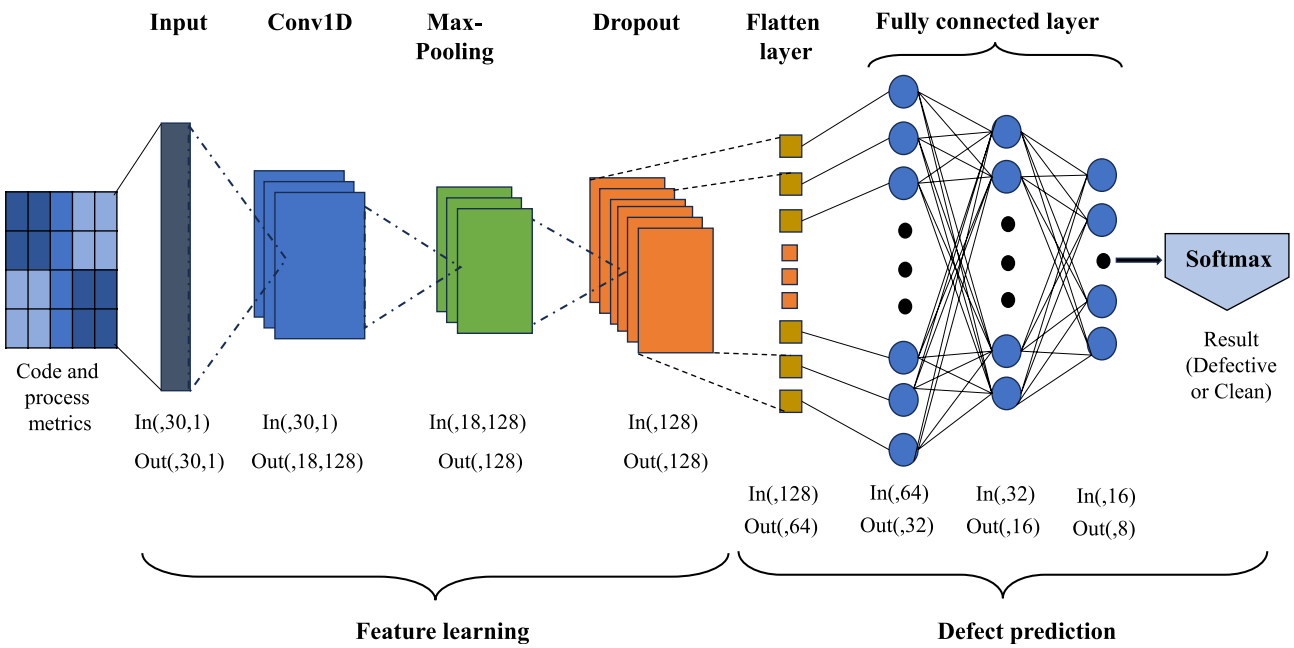

Fig. 3. Architecture of a deep CNN in the DMLM learning model.

Projects	Android version	Total classes	Total LOC	Defective rate %
TV	Android-s-beta5	524	101266	38.74
	Android13	761	158664	40.87
Messaging	Android-s-beta-4	526	121611	63.31
	Android13	526	121644	63.50
Contacts	Android-s-beta-4	425	94151	42.35
	Android12-dev	423	93887	42.08
Gallery	Android10	521	102174	29.75
	Android11	509	98699	28.68
Documents UI	Android-s-beta-5	473	78559	39.96
	Android13	475	79692	40.42
Bluetooth	Android-mainline10	286	114531	59.79
	Android10-s1	280	112518	59.64
Browser	Android-cts-6	150	40134	40
	Android4	158	44537	41.14
Camera	Android-sdk-4	92	19302	23.91
	Android-6	105	24187	28.57
Email	Android-s-10	246	70719	33.74
	Android-8	245	70645	33.06

Table 1. Android projects description.

Features extracting

In this step, two main kinds of Android app metrics are extracted: Code metrics and process metrics. Specifically, 20 essential code metrics are collected, including LOC, a measure of functional abstraction (MFA), etc., as outlined in Table 2. Additionally, ten significant process metrics are extracted from the Android projects, such as lines of code added (LA), lines of code deleted (LD), etc., as documented in Table 3. The Version Control System (VCS), Git, was utilized to extract these metrics by providing comprehensive historical data on code changes. By analyzing commit metadata and code modifications, VCS facilitates tracking developer contributions, code churn, and overall project activity. This data not only helps assess code quality and stability but also supports the identification of trends and patterns that impact defect prediction and project outcomes. These hybrid metrics provide a comprehensive view of Android projects, allowing us to develop a robust and accurate mobile app defect prediction method that combines static code features with code change metrics.

Two essential Python libraries, re (https://docs.python.org/3/library/re.html.) and Javalang (https://github.com/c2nes/javalang.), were also applied to extract metrics from the collected Android projects. These libraries

Feature	Description	Feature	Description
MFA	Measure of Functional Abstraction.	CBO	Coupling Between Object classes.
CAM	Cohesion Among Methods of Class.	RFC	Response for a Class.
WMC	Weighted Methods per Class.	NPM	The number of Public Methods.
DIT	Depth of Inheritance Tree.	DAM	Data Access Metric.
NOC	Number of Children.	MOA	Measure of Aggregation.
LOC	Lines of Code.	CE	Efferent Couplings.
LCOM	Lack of Cohesion in Methods.	IC	Inheritance Coupling.
LCOM3	Lack of Cohesion in Methods, different from LCOM.	CBM	Coupling Between Methods.
Max CC	Maximum value of CC methods of the investigated class	AMC	Average Method Complexity.
Avg CC	The arithmetic mean of the CC value in the investigated class	CA	Afferent Couplings.

Table 2. Description of the code metrics.

Feature	Description	Feature	Description
NS	Number of modified subsystems	LA	Lines of code added
ND	Number of modified directories	LD	Lines of code deleted
NF	Number of modified files	LT	Lines of code in a file before the change
Entropy	Number of modified code across each file	NDEV	Number of developers working on the files
FIX	Whether or not the change is a bug fix	NUC	Number of unique changes to modified files.

Table 3. Description of the process metrics.

played a crucial role in enabling us to analyze the source code and retrieve valuable information related to the software's design and functionality. The re library, which stands for regular expressions, provided a powerful tool for pattern matching and string manipulation. Specific source code patterns corresponding to various code metrics were deeply identified by leveraging regular expressions. On the other hand, the Javalang library is a Java 8 parser and lexer written in Python that provides an easy way for parsing and analyzing Java code.

After the extraction of these metrics, we then analyzed the individual correlation between each of these metrics and the defect label. Figure 4 presents correlation heatmaps illustrating the relationships between the extracted metrics and defect labels. These visualisations help to observe clearly which features have a stronger predictability relationship with defect-prone modules. As shown in Fig. 4, code metrics such as WMC and LOC are strongly positively correlated with their respective defect labels. These positive values indicate that higher complexity, coupling, and size are more likely to cause defects. On the other hand, metrics such as CAM and DIT are weakly or negatively correlated with their respective defect labels, indicating that they have a limited direct influence on defect prediction. As to process metrics, metrics like LA and LD, and Entropy, are highly positively correlated with defect-prone modules. Their positive values would suggest that persistent and far-reaching code changes cause an increase in one's defectiveness to instability (making them excellent indicators of potential defects). On the other hand, metrics such as NDEV and FIX exhibit moderate positive or negative correlations, indicating that their direct impact on defect prediction is limited, as evidenced by their relatively small correlation coefficients with the respective defect labels.

When comparing Tables 2 and 3 with the heatmaps in Fig. 4, it demonstrates that integrating high-impact code metrics (e.g., WMC, LOC) with dynamic process metrics (e.g., LA, LD, Entropy) provides a more comprehensive understanding of defect patterns. This approach reinforces the rationale for using a hybrid metrics approach in mobile App defect prediction, as the two metric types capture complementary aspects of mobile App quality.

Learning model and defect prediction

The proposed deep learning architecture comprises two primary components: a CNN module and a sequence of fully connected layers. As depicted in Fig. 3, the CNN module comprises an input layer that processes the extracted features, followed by a one-dimensional convolutional layer (Conv1D) designed to capture spatial dependencies within the data. Subsequently, a global max-pooling layer is incorporated to reduce dimensionality while retaining the most relevant feature representations. A dropout layer is employed as a regularization technique to mitigate the risk of overfitting and enhance generalization. Finally, the module includes a flattened layer, which transforms the pooled feature maps into fully connected layers, facilitating integration with subsequent layers for prediction tasks. Rectified Linear Unit (ReLU) activation functions are applied across all layers except the flattened layer, which employs a sigmoid activation function. The subsequent fully connected component incorporates three densely connected hidden layers.

In the final stage of the DMLM model, the classification process determines whether a mobile app module is defective or clean. A softmax activation function facilitates this prediction by transforming the raw outputs into normalized probability scores corresponding to the two classification categories. From a mathematical perspective, this relationship can be formulated as follows:

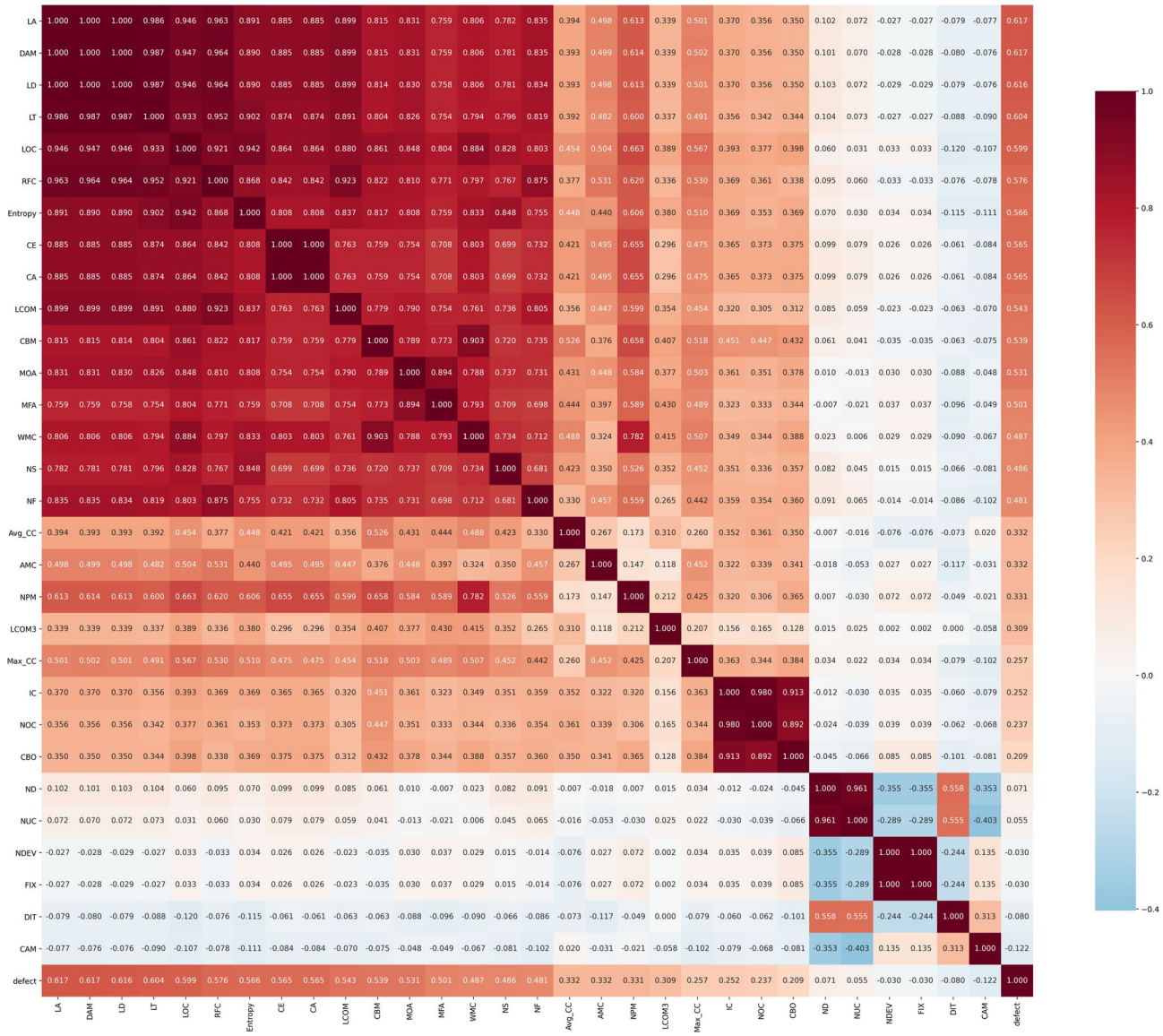

Fig. 4. Correlation heatmap for code and process metrics in the used dataset.

$$\mathrm{y} = \mathrm{softmax}(Wx + b), \tag{1}$$

where y represents the vector of predicted probabilities, w is the weight matrix, x denotes the input vector, and b corresponds to the bias vector. The softmax function transforms the raw output scores into probability values collectively sum to 1. The class associated with the highest probability is then selected as the final prediction.

We employ binary cross-entropy as the loss function to train the model, which quantifies the disparity between the predicted probability distribution and the actual class labels. A lower loss value signifies a more accurate model. The binary cross-entropy loss is mathematically defined as:

$$loss = -\left(ytrue * \log\left(ypred\right) + \left(1 - ytrue\right) * \log\left(1 - ypred\right)\right), \tag{2}$$

where y_true is the true binary labels and y_pred is the predicted probabilities. The Adam optimization algorithm is utilized to effectively adjust the model's weights and biases, aiming to minimize the loss function. By integrating principles from both AdaGrad and RMSProp, Adam dynamically adapts the learning rate throughout the training process, enhancing optimization efficiency. This procedure helps the model converge faster to an optimal set of parameters. In summary, the output layer makes predictions using a softmax function, is trained via cross-entropy loss, and uses the Adam optimizer to learn the model parameters. This standard setup allows the model to perform binary defect classification effectively.

Experimental setup

To evaluate the effectiveness of the DMLM model, we conducted a series of experiments, benchmarking its performance against six state-of-the-art methodologies. The following subsections provide a detailed overview of the key experimental procedures and configurations.

Research questions

In this paper, we present our experimental results based on these research questions:

RQ1: How practical is the DMLM approach compared with other baseline methods under non-effort-aware conditions?
RQ2: How does the effectiveness of the DMLM model compare to other baseline methods in effort-aware conditions?
RQ3: How does the proposed DMLM approach statistically surpass other models?

Baseline methods

In this paper, we compare the effectiveness of the DMLM model with six state-of-the-art mobile app defect prediction methods as follows:

MTL-DNN[13]: MTL-DNN is a multitask deep neural network method designed to address mobile app defect prediction.

SDF[11]: SDF is a simplified deep forest model designed for the prediction of Android app defects based on a modified random forest algorithm that eliminates the multi-grained scanning operation commonly used for high-dimensional data.

KAL[12]: KAL is a defect prediction framework for Android mobile apps that uses kernel-based principal component analysis to transform features and adversarial learning to extract standard feature embeddings.

CDFE[15]: CDFE is a cross-triplet deep feature embedding method for cross-app JIT bug prediction, integrating a cross-triplet loss function into a deep neural network to enhance feature representation and reduce discrepancies in cross-app bug data.

DNN[7]: DNN is a deep neural network-based defect prediction model for Android APKs, utilizing smali2vec to extract token and semantic features from decompiled smali files to enhance predictive accuracy.

ML[8]: ML is a machine learning-based defect prediction model for Android apps that leverages object-oriented metrics to identify defective classes.

To provide a more comprehensive and detailed comparison of the baseline methods evaluated in this study, Table 4 offers a side-by-side view of each baseline's architectural design, input features, evaluation scope, advantages, and limitations, thereby facilitating clearer insight into their respective strengths and applicability for Android app defect prediction.

Handling imbalance

In software defect data, class imbalance is a prevalent challenge. Defective instances typically constitute a small proportion of the overall instances, leading to varying imbalance ratios depending on the defective rate percentage. Among the projects listed in Table 1, Camera-Android-sdk-4 exhibits the highest level of imbalance, with a defect rate of 23.91%. Imbalanced data can significantly impact the model's predictive capabilities, particularly in accurately identifying defect instances, ultimately leading to lower performance[21–23]. Prior works have proposed two popular approaches to mitigate the impact of class imbalance. The first method involves oversampling[24–26],

Study	Architectural design	Input feature	Evaluation scope	Advantages	Limitations
SDF	Simplified Deep Forest (cascade ensemble forests, no multigrain scanning) for JIT defect prediction in Android Apps	6 process metrics (LA, LD, NF, ND, NUC, NDEV)	10 Android apps, F1, MCC, AUC; Wilcoxon and Cliff's delta for statistical evaluation	Handles low-dimensional input well; no tuning needed for cascades	Not designed for high-dimensional data; ignores complexity, size, code structure; only Android Apps
MTL-DNN	Multi-task DNN with shared + task-specific layers for JIT defect prediction	14 process metrics (LA, LD, LT, NS, ND, NF, Entropy, FIX, EXP, REXP, SEXP, NDEV, AGE, NUC)	15 Android apps, compared with DNN, SVM, RF; F1, MCC; Wilcoxon and Cliff's delta; 30×3-fold CV	Works well when data is scarce; robust with little labeled data	Resource-intensive; only app-level tasks; tuning required for dissimilar apps; only Android Apps
KAL	Kernel PCA + Adversarial Learning for JIT defect prediction	6 process metrics (NUC, NDEV, LA, LD, NF, ND)	14 Android apps, effort-aware metrics (EARecall, EAF, Popt, PCI)	Combines nonlinear mapping and domain invariance	Not evaluated on code structure features; only Android Apps
CDFE	Cross-triplet Deep Feature Embedding (DNN with cross-triplet loss) for defect prediction	14 process metrics (LA, LD, LT, NS, ND, NF, Entropy, FIX, EXP, REXP, SEXP, NDEV, AGE, NUC)	19 Android apps, effort-aware (EARecall, EAF)	Adapts triplet loss for cross-app/domain data; outperforms ML methods	Needs more labeled data; depends on feature engineering; not generalizable to all app types
DNN	Deep Neural Network using smali2vec (token-based features from APKs)	Token-based features from smali files (decompiled APKs)	10 Android apps; ROC-AUC, F1, error rate, time/space cost	Can utilize binaries; does not require source code	Requires high resources; noisy predictions from binaries
Proposed (DMLM)	Deep Multi-Metrics Learning Model (deep CNN combining code + process metrics)	20 code metrics (LOC, CBO, WMC, etc.) and 10 process metrics (LA, LD, LT, NDEV)	9 Android apps, multiple releases; metrics: F1, AUC, MCC, PofB20	Captures static structure + historical evolution; direct fusion of code + process features; outperforms baselines	Only Android apps; not tested on iOS; requires historical release data

Table 4. Comprehensive comparison of the proposed approach and baseline methods.

duplicating training instances from the minority class. The second method, undersampling[27,28], reduces the number of training instances from the majority class. In this work, we choose the undersampling technique in case the defective rate is more than 50%, like the Messaging-Android-s-beta-4, and used the oversampling method in case the defective rate is less than 50%, like the Camera-Android-sdk-4.

Performance indicators

Four key performance metrics were utilized to evaluate the proposed DMLM model under non-effort and effort-aware settings. F1 scores, AUC, and MCC were used as measures for non-effort-aware scenario evaluation. For effort-aware scenario evaluation, the PofB20 metric was used.

Non-effort-aware evaluation measures

In this scenario, we assume that the available resources are adequate for conducting defect analysis based on the predictions generated by the defect prediction model. This implies that all instances identified as defective can be thoroughly examined. To demonstrate the effectiveness of our approach in a non-effort-aware setting, we utilize F1 scores, AUC, and MCC as key performance evaluation metrics. The following are the detailed definitions:

$$\mathrm{F1} = \frac{2 * (\ \mathrm{Precision}\ *\ \mathrm{Recall}\ )}{(\ \mathrm{Precision}\ +\ \mathrm{Recall}\ )}, \tag{3}$$

where

$$\mathrm{Recall}\ = \frac{TP}{(\mathrm{TP} + \mathrm{FN})} \tag{4}$$

$$\mathrm{Precision}\ = \frac{TP}{(TP + FP)}. \tag{5}$$

AUC is a standard metric for binary classifiers and quantifies model performance by measuring the area under the ROC curve, which depicts the trade-off between the true positive rate (TPR) and the false positive rate (FPR). The AUC is determined using the following equation:

$$\mathrm{AUC} = \ \mathrm{integral}(\mathrm{TPR},\ \ \mathrm{FPR})\ . \tag{6}$$

MCC is a specific instance of the Matthews correlation coefficient, offering a holistic assessment by considering TP, TN, FP, and FN. Its mathematical expression is as follows:

$$MCC = \frac{(TP * TN) - (FP * FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \tag{7}$$

True Positives (TP) represent the number of defective files the model correctly identifies as defective. Accurately detecting these defective files is crucial for ensuring software reliability, as failing to do so may result in undetected bugs affecting the system. On the other hand, False Positives (FP) occur when the model incorrectly classifies defect-free files as defective. Many false positives can lead to unnecessary debugging efforts, wasting valuable resources and time for developers. Meanwhile, False Negatives (FN) refer to defective files that the model fails to recognize, incorrectly classifying them as defect-free. False negatives can be particularly problematic, as undetected defects may lead to software failures, negatively impacting user experience and system performance.

Effort-aware evaluation measures

Effort-aware scenarios come into play when there are constraints on testing resources or looming project deadlines, a common scenario in real-life defect prediction scenarios. In such situations, we often can thoroughly examine only a limited number of predicted buggy instances. Consequently, assessing prediction performance using tailor-made metrics is essential when dealing with an effort-aware context. This study adopts PofB20 as the evaluation metric within the effort-aware context[29].

PofB20[29,30] serves as a metric to assess the proportion of faults a developer can uncover by inspecting 20% of LOC. When the developer completes the inspection of 20% of the LOC within the testing data, the PofB20 scores represent the percentage of bugs detected through the inspection results. The PofB20 metric is bounded within the range [0, 1], with higher values indicating more effective model performance. The computation of PofB20 involves the following steps: Initially, instances in the test files are sorted based on the confidence levels, which are the probabilities assigned by a prediction model to each instance, indicating the likelihood of it being defective. Instances with higher confidence levels are considered more likely to be defective. Subsequently, during this process, we collect data on the inspected lines of code and the quantity of identified defects. Once 20% of the LOC within the test data has been inspected, the process terminates, and the resulting percentage of identified bugs is defined as the PofB20 score. A higher PofB20 score signifies a greater ability to detect defects by examining a limited portion of the LOCs.

Results analysis and discussion

The design of the experiments conducted in this study, along with the subsequent analysis and discussion of the results, is structured to demonstrate that integrating code metrics and process metrics using a deep CNN yields

Projects	F1 scores			AUC scores			MCC scores			PofB20 scores		
	Code	Process	Both	Code	Process	Both	Code	Process	Both	Code	Process	Both
TV	0.608	0.598	0.614	0.687	0.690	0.701	0.403	0.422	0.451	0.299	0.301	0.334
Messaging	0.547	0.553	0.587	0.805	0.800	0.836	0.483	0.487	0.522	0.427	0.424	0.482
Contacts	0.480	0.482	0.489	0.478	0.466	0.487	0.412	0.418	0.463	0.309	0.316	0.393
Gallery	0.497	0.509	0.574	0.710	0.699	0.758	0.406	0.433	0.496	0.380	0.377	0.414
Documents UI	0.507	0.490	0.533	0.516	0.511	0.573	0.325	0.309	0.382	0.307	0.298	0.341
Bluetooth	0.656	0.681	0.710	0.813	0.796	0.851	0.491	0.507	0.550	0.311	0.342	0.369
Browser	0.398	0.400	0.422	0.620	0.611	0.684	0.196	0.200	0.202	0.485	0.500	0.538
Camera	0.388	0.411	0.445	0.614	0.600	0.659	0.329	0.313	0.367	0.300	0.319	0.350
Email	0.702	0.684	0.723	0.792	0.773	0.815	0.360	0.357	0.368	0.293	0.287	0.302
AVG	0.531	0.534	0.566	0.671	0.661	0.707	0.378	0.383	0.422	0.346	0.352	0.391

Table 5. F1 scores, AUC, MCC, and PofB20 scores of DMLM for code metrics, process metrics, and both metrics.

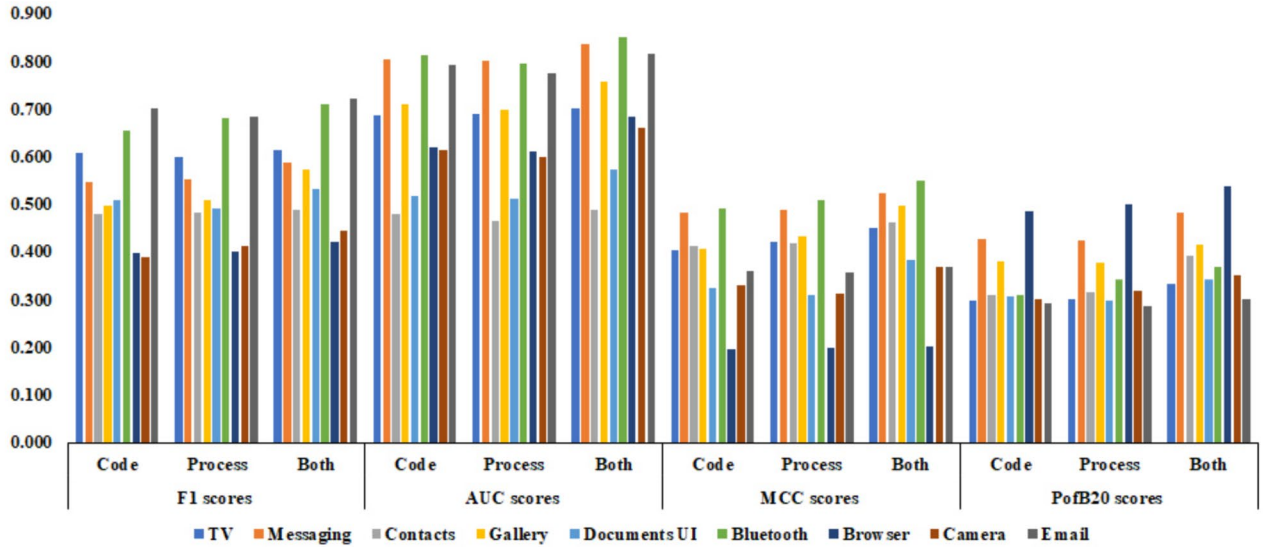

Fig. 5. F1 scores, AUC, MCC, and PofB20 scores of DMLM for code metrics, process metrics, and both metrics.

superior mobile App defect prediction compared to using either metric type in isolation. As shown in Table 5 and Fig. 5, the experimental design proceeds in three stages to evaluate the contribution of each metric type.

In the first phase, only code metrics such as LOC and CBO extracted from the mobile app are used to train and evaluate the model. Table 5 presents the average performance indicators at this stage, with F1 scores of 0.531, AUC of 0.671, MCC of 0.378, and PofB20 of 0.346.

In the second phase, the model is trained and evaluated using only process metrics derived from historical code changes, such as LD and LA. The performance was similar or slightly lower than code metrics alone, with averages marginally differing: F1 scores 0.534, AUC 0.661, MCC 0.383, and PofB20 0.352.

In the final phase, both static code and process metrics are integrated into the model using deep CNNs to capture complex interactions and complementary information. This fusion enhances mobile App defect prediction performance significantly over individual sets. As shown in Table 5 and Fig. 5, the average F1 score rises to 0.566, AUC to 0.707, MCC to 0.422, and PofB20 to 0.391, which confirms that the complementary information provided by both metric sets leads to more accurate defect prediction.

Figure 5 also presents a visual representation of these improvements, showing that performance is enhanced in both metric types. This approach makes the DMLM model benefit significantly from a mixture of characteristics to improve the defect prediction performance in mobile apps.

The evaluation process of the proposed DMLM, compared with state-of-the-art methods, is structured around addressing the research questions outlined in Section 5.1. The predictive performance of the proposed DMLM model is assessed through a comparative analysis against six state-of-the-art methods: ML, DNN, CDFE, KAL, SDF, and MTL-DNN. The evaluation is conducted under non-effort-aware conditions using F1 score, AUC, and MCC, and under effort-aware conditions using PofB20.

Projects	ML	DNN	CDFE	KAL	SDF	MTL-DNN	DMLM
TV	0.539	0.509	0.511	0.525	0.605	0.562	**0.614**
Messaging	0.479	0.548	**0.610**	0.522	0.401	0.550	0.587
Contacts	0.426	0.399	0.400	0.464	0.470	0.487	**0.489**
Gallery	0.436	0.325	0.377	0.473	0.386	0.354	**0.574**
Documents UI	0.505	0.508	0.586	0.550	**0.588**	0.546	0.533
Bluetooth	0.677	0.652	0.619	0.713	0.634	**0.718**	0.710
Browser	0.404	0.393	0.407	0.362	0.419	0.403	**0.422**
Camera	0.395	0.430	0.425	0.369	0.437	0.308	**0.445**
Email	0.533	0.636	0.572	0.488	0.639	0.548	**0.723**
AVG	0.492	0.489	0.501	0.496	0.508	0.497	**0.566**

Table 6. F1 scores of ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under within projects.

Projects	ML	DNN	CDFE	KAL	SDF	MTL-DNN	DMLM
TV	0.660	0.665	0.626	0.691	0.684	**0.711**	0.701
Messaging	0.663	0.672	0.713	0.772	0.608	0.622	**0.836**
Contacts	0.467	0.509	**0.521**	0.506	0.478	0.483	0.487
Gallery	0.621	0.587	0.579	0.603	0.642	0.698	**0.758**
Documents UI	0.543	0.548	0.572	0.476	0.558	0.560	**0.573**
Bluetooth	0.739	0.805	0.814	0.721	0.812	0.727	**0.851**
Browser	0.736	**0.748**	0.707	0.722	0.651	0.584	0.684
Camera	0.665	0.650	0.629	**0.684**	0.579	0.593	0.659
Email	0.684	0.679	0.633	0.600	0.788	0.771	**0.815**
AVG	0.642	0.651	0.644	0.642	0.644	0.639	**0.707**

Table 7. AUC scores of ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under within projects.

Projects	ML	DNN	CDFE	KAL	SDF	MTL-DNN	DMLM
TV	0.381	0.408	0.387	0.421	0.412	0.377	**0.451**
Messaging	0.388	0.395	0.436	0.493	**0.534**	0.459	0.522
Contacts	0.383	0.472	**0.494**	0.400	0.305	0.394	0.463
Gallery	0.244	0.263	0.304	0.386	0.482	0.480	**0.496**
Documents UI	0.337	0.369	0.354	0.339	0.358	0.353	**0.382**
Bluetooth	0.489	0.452	0.443	0.508	0.483	0.450	**0.550**
Browser	0.240	0.195	0.263	**0.244**	0.132	0.181	0.202
Camera	0.366	0.328	0.262	0.211	0.221	0.229	**0.367**
Email	0.339	0.367	0.362	0.345	0.318	0.365	**0.368**
AVG	0.352	0.361	0.367	0.372	0.361	0.365	**0.422**

Table 8. MCC scores of ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under within projects.

To address RQ1, a comprehensive performance comparison is conducted between the proposed DMLM model and six baseline methods. Table 6 illustrates the evaluation results based on F1 scores, highlighting the predictive effectiveness of each approach. Similarly, Table 7 presents the comparative analysis regarding AUC, providing insights into the models' ability to distinguish between defective and non-defective instances. Additionally, Table 8 showcases the performance assessment using MCC, further demonstrating the reliability and robustness of the proposed model compared to existing techniques. These experimental findings indicate that the DMLM approach outperforms other baseline approaches in the context of non-effort-aware mobile app defect prediction. Specifically, DMLM exhibited superior performance across multiple evaluation metrics, including F1 scores, AUC, and MCC. The average F1 scores achieved by DMLM stood at 0.566, surpassing the corresponding scores of all baseline methods. The average AUC score for DMLM was notably higher at 0.707 compared to the AUC scores of the baseline methods. Moreover, the average MCC scores of DMLM reached 0.422, consistently outperforming the MCC scores associated with the baseline methods. These results show that fusing code and process metrics can enhance a mobile app defect prediction under non-effort aware scenarios.

Figures 6, 7, and 8 provide a visual comparison of the performance distribution of the proposed DMLM model against six baseline methods (ML, DNN, CDFE, KAL, SDF, and MTL-DNN) across all seven experiments.

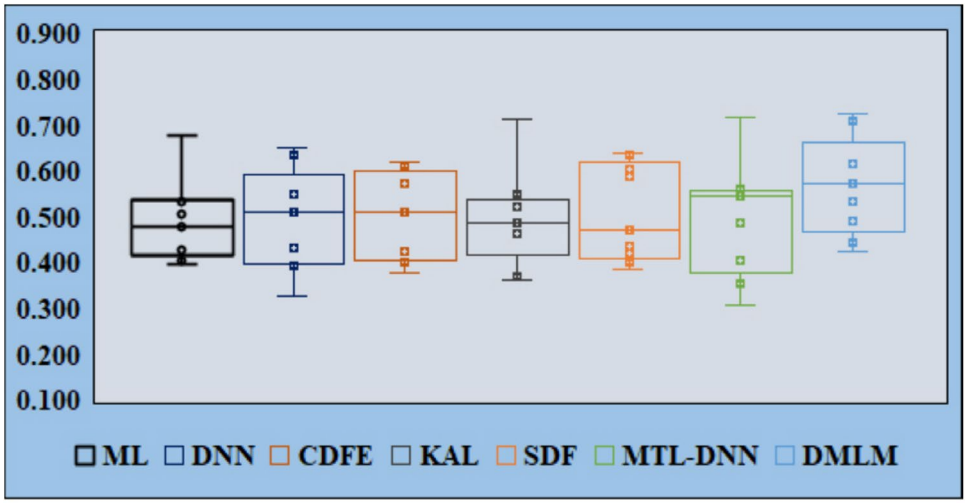

Fig. 6. Comparison of F1 scores for ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under within projects.

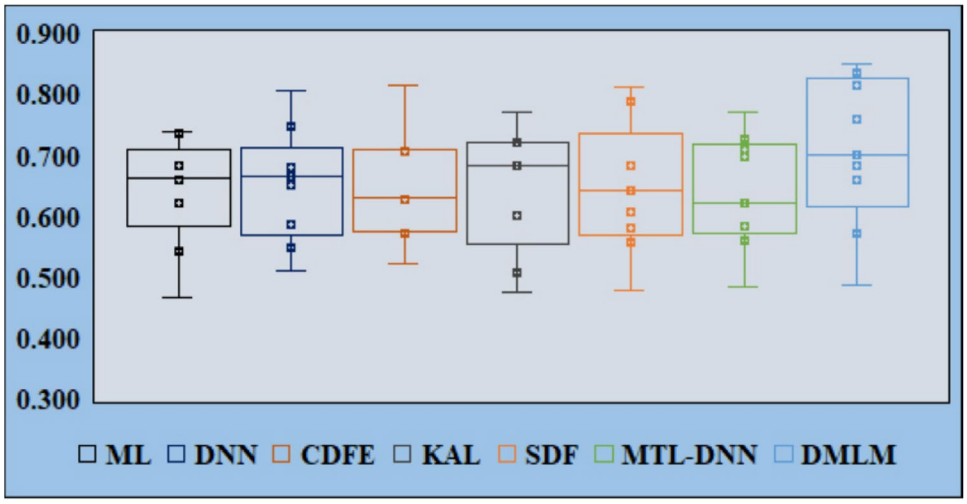

Fig. 7. Comparison of AUC scores for ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under within projects.

Specifically, Fig. 6 illustrates the boxplots of F1 scores for each approach based on the results from Table6, while Fig. 7 presents the AUC boxplots corresponding to the evaluations in Table7. Likewise, Fig. 8 depicts the MCC boxplots derived from Table8. Each boxplot represents the distribution of F1 scores, AUC, and MCC across the experiments, capturing key statistical measures such as the median, upper, and lower quartiles for all evaluated models. The visual analysis demonstrates that the DMLM model consistently performs better than the six baseline methods in terms of F1 scores, AUC, and MCC across most tasks. Overall, the results confirm that the proposed DMLM model outperforms existing state-of-the-art approaches in mobile app defect prediction under non-effort-aware conditions. These findings highlight the effectiveness and robustness of DMLM in capturing critical defect-related patterns and improving predictive accuracy in mobile App.

In addition, the evaluation and comparative process is extended to cross-project scenarios, where the prediction model is first trained using sufficient existing training data on one project containing defect labels. Then, this trained model will predict the defects of another new project with insufficient labeled data. As shown in Table 9 DMLM achieved the highest average F1 score of 0.465, superior to all baseline approaches. This illustrates that DMLM effectively transfers learned patterns across different projects. Table 10 shows that the average AUC for DMLM was 0.574, indicating an outperforming defect prediction performance compared to baseline methods, which scored below 0.52 on average. DMLM also achieved the highest MCC (0.362) as presented in Table 11, demonstrating efficiency and consistent predictions even in cross-project contexts.

The boxplots presented in Figs. 9, 10, 11 demonstrate that DMLM not only achieves a higher median performance but also exhibits a more stable and consistent distribution of scores across various project pairs

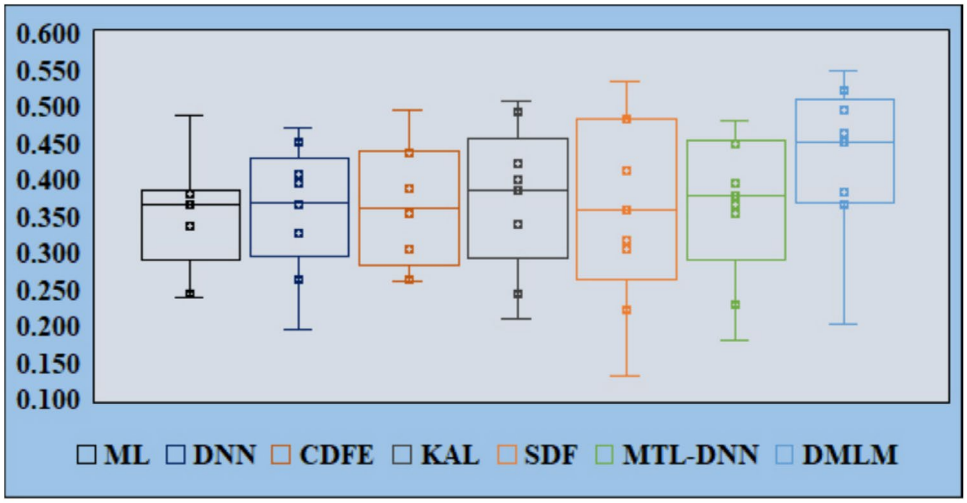

Fig. 8. Comparison of MCC scores for ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under within projects.

Projects		ML	DNN	CDFE	KAL	SDF	MTL-DNN	DMLM
Source	Target							
Messaging	Browser	0.373	0.443	0.393	0.429	0.482	0.407	**0.504**
TV	Email	0.311	0.407	0.470	0.427	0.399	0.441	**0.482**
Contacts	Bluetooth	0.326	0.300	0.308	0.380	0.375	**0.412**	0.402
Gallery	Camera	0.334	0.245	0.290	0.387	0.324	0.359	**0.471**
Documents UI	Email	0.386	0.383	**0.451**	0.450	0.439	0.411	0.438
Messaging	Bluetooth	0.462	0.491	0.477	0.557	0.505	0.567	**0.583**
Bluetooth	Browser	0.309	0.296	0.313	0.296	0.334	0.318	**0.373**
AVG		0.357	0.366	0.386	0.418	0.408	0.416	**0.465**

Table 9. F1 scores of ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under cross projects.

Projects		ML	DNN	CDFE	KAL	SDF	MTL-DNN	DMLM
Source	Target							
Messaging	Browser	0.505	0.501	0.482	0.565	0.545	0.562	**0.576**
TV	Email	0.507	0.506	0.549	0.631	0.485	0.515	**0.686**
Contacts	Bluetooth	0.357	0.383	0.401	**0.414**	0.381	0.382	0.400
Gallery	Camera	0.475	0.442	0.446	0.493	0.512	0.551	**0.622**
Documents UI	Email	0.415	0.413	0.440	0.389	0.445	0.442	**0.470**
Messaging	Bluetooth	0.565	0.606	0.627	0.590	0.647	0.574	**0.699**
Bluetooth	Browser	0.563	0.563	0.544	0.474	0.519	**0.591**	0.562
AVG		0.484	0.488	0.498	0.508	0.505	0.517	**0.574**

Table 10. AUC scores of ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under cross projects.

compared to the baseline methods. These findings suggest that DMLM's fused feature representation effectively captures transferable patterns of defect proneness relevant to different mobile application domains. These findings confirm that DMLM generalizes well beyond the training project, making it a practical tool for predicting defects in projects that have minimal historical defect data.

For RQ2, the effort-aware evaluation aims to rank the probability density of defects based on the predicted probability value and the LOC. To conduct this evaluation, all experiments are performed, and the PofB20 of each experiment is calculated according to the setup details outlined in Section 5.4. The performances of the proposed DMLM model and other baseline methods are then compared using PofB20 under effort-aware conditions. Table 12 displays the performance of the proposed DMLM model against these six baseline methods in terms of PofB20. The results showed the superior performance of the DMLM approach in effort-aware mobile app defect prediction. Specifically, DMLM exhibited notable effectiveness in the PofB20 measure, surpassing the

Projects		ML	DNN	CDFE	KAL	SDF	MTL-DNN	DMLM
Source	Target							
Messaging	Browser	0.291	0.307	0.298	0.314	0.328	0.298	**0.370**
TV	Email	0.297	0.297	0.336	0.386	0.426	0.363	**0.429**
Contacts	Bluetooth	0.293	0.355	0.380	0.327	0.243	**0.404**	0.393
Gallery	Camera	0.187	0.198	0.234	0.316	0.384	0.379	**0.407**
Documents UI	Email	0.258	0.278	0.273	0.277	0.285	0.279	**0.314**
Messaging	Bluetooth	0.374	0.340	0.341	0.406	0.385	0.356	**0.452**
Bluetooth	Browser	0.184	0.147	**0.203**	0.200	0.105	0.143	0.166
AVG		0.269	0.275	0.295	0.318	0.308	0.317	**0.362**

Table 11. MCC scores of ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under cross projects.

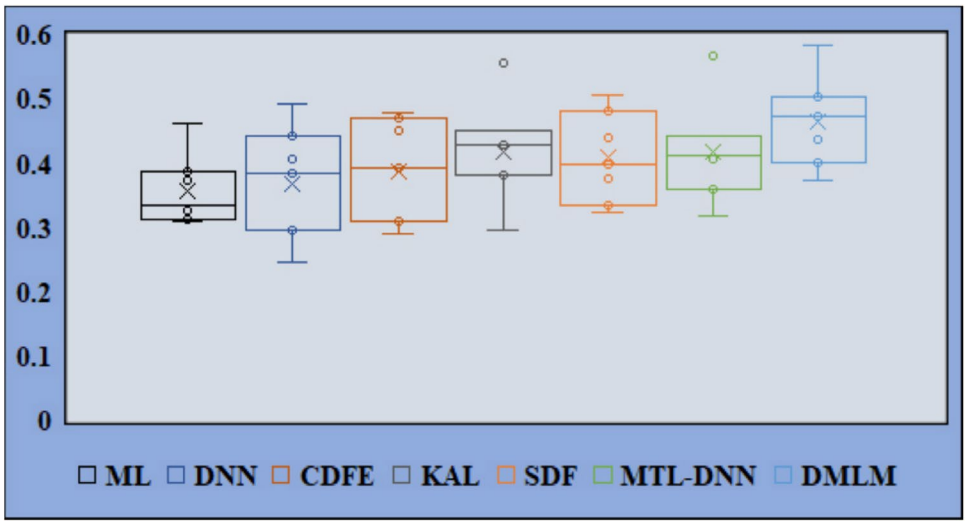

Fig. 9. Comparison of F1 scores for ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under cross projects.

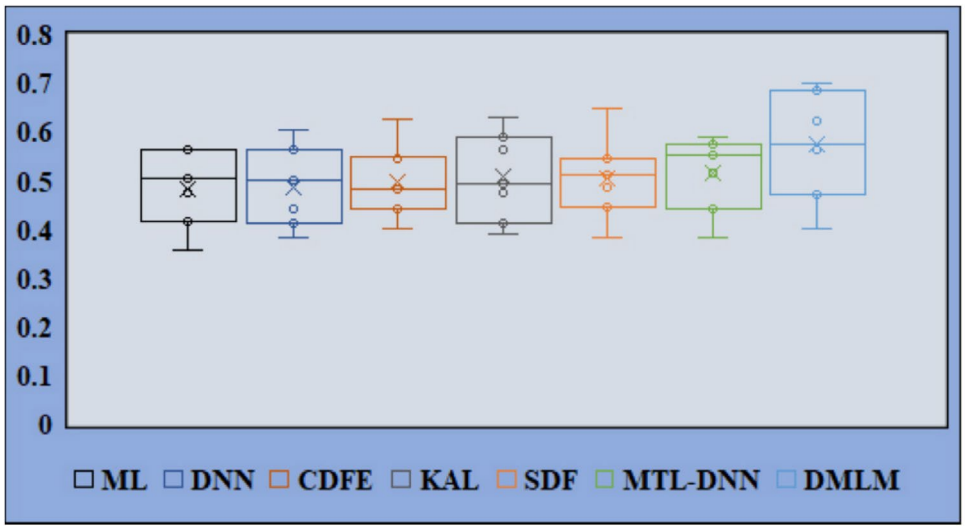

Fig. 10. Comparison of AUC scores for ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under cross projects.

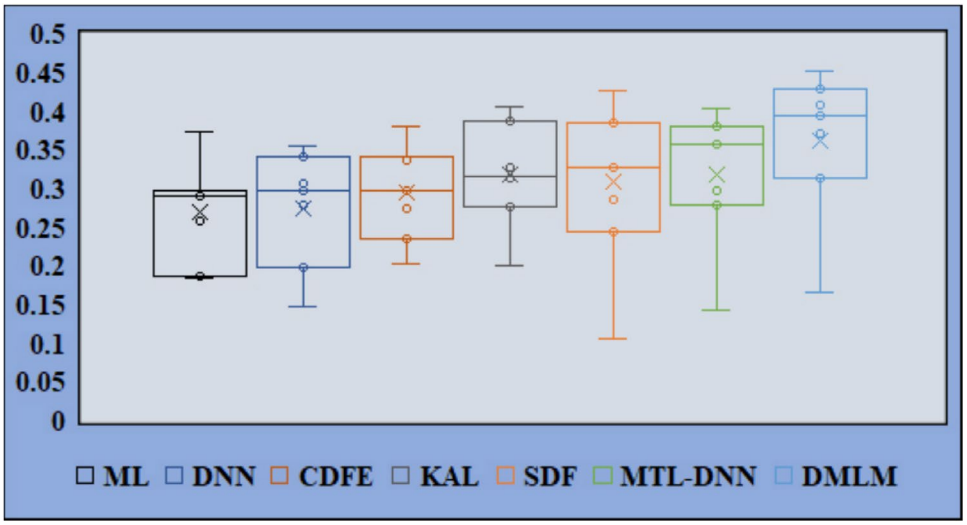

Fig. 11. Comparison of MCC scores for ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under cross projects.

Projects	ML	DNN	CDFE	KAL	SDF	MTL-DNN	DMLM
TV	0.323	0.311	0.347	**0.351**	0.299	0.293	0.334
Messaging	0.425	0.358	0.404	0.420	0.397	0.318	**0.482**
Contacts	0.379	0.366	0.300	0.338	0.384	0.359	**0.393**
Gallery	0.262	0.329	0.400	0.385	0.344	0.386	**0.414**
Documents UI	0.216	0.201	**0.374**	0.354	0.270	0.213	0.341
Bluetooth	0.311	0.286	0.264	0.271	**0.380**	0.361	0.369
Browser	0.515	0.513	0.472	0.437	0.490	**0.539**	0.538
Camera	0.246	0.233	0.342	0.315	0.285	0.275	**0.350**
Email	0.263	0.299	0.284	0.277	0.286	0.298	**0.302**
AVG	0.327	0.322	0.354	0.350	0.348	0.338	**0.391**

Table 12. PofB20 scores of ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under within projects.

capabilities of state-of-the-art baseline methods. DMLM achieved a PofB20 score of 0.391, clearly outperforming the corresponding scores of all the baseline methods considered in the study. These results underscore the efficacy of the DMLM approach in enhancing the accuracy of mobile app defect prediction in scenarios where the consideration of development effort is paramount. In addition, Fig. 12 provides a comparative analysis of the PofB20 metric for the proposed DMLM model and six baseline methods (ML, DNN, CDFE, KAL, SDF, and MTL-DNN) across seven experimental settings, as detailed in Table 12. The boxplots illustrate the distribution of PofB20 values, capturing key statistical indicators such as the median, upper quartile, and lower quartile for each approach. The visual representation in Fig. 12 highlights the superior performance of DMLM compared to the baseline methods in nearly all experimental cases. The consistently higher PofB20 values achieved by DMLM demonstrate its effectiveness in identifying defective components while considering testing effort constraints. Overall, the results confirm that the proposed DMLM model outperforms existing state-of-the-art approaches in effort-aware mobile app defect prediction. These outcomes underscore the model's capability to optimize defect detection while efficiently allocating testing resources, making it a more reliable solution for practical software quality assurance.

In addition, the evaluation and comparative process is extended in the cross-project assessment under effort-aware conditions, using the PofB20 metric, which further solidifies the practical value of DMLM. As shown in Table 13, DMLM achieves the highest average PofB20 score of 0.347, indicating that it helps developers to identify the most defects by inspecting only 20% of the code in a new, unseen project. This performance outperforms all baseline methods. Figure 13 demonstrates the superiority of DMLM, as it shows that the PofB20 values for DMLM are consistently higher across different project transfer pairs. This finding is significant because it confirms that DMLM can effectively prioritize inspection efforts even when used on a new project with training data from another project. These findings demonstrate that the proposed approach is a valuable tool for development environments with limited resources.

For RQ3, we employ the Wilcoxon Signed Rank Test (WSRT) and Cliff's delta to determine whether the performance differences between the proposed DMLM model and the baseline models are statistically significant.

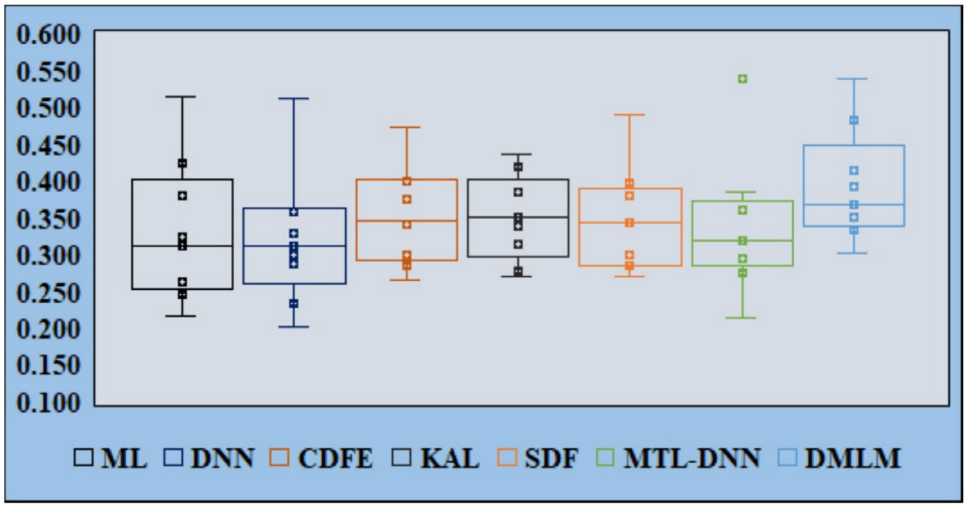

Fig. 12. Comparison of PofB20 scores for ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under within projects.

Projects		ML	DNN	CDFE	KAL	SDF	MTL-DNN	DMLM
Source	Target							
Messaging	Browser	0.247	0.234	0.267	**0.287**	0.238	0.231	0.274
TV	Email	0.325	0.270	0.311	0.344	0.316	0.251	**0.396**
Contacts	Bluetooth	0.290	0.276	0.231	0.276	0.306	0.284	**0.323**
Gallery	Camera	0.200	0.248	0.308	0.315	0.274	0.305	**0.340**
Documents UI	Email	0.165	0.151	**0.295**	0.269	0.215	0.168	0.280
Messaging	Bluetooth	0.238	0.215	0.233	0.222	0.303	0.285	**0.333**
Bluetooth	Browser	0.394	0.386	0.363	0.357	0.391	0.426	**0.482**
AVG		0.266	0.254	0.287	0.296	0.292	0.279	**0.347**

Table 13. PofB20 scores of ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under cross projects.

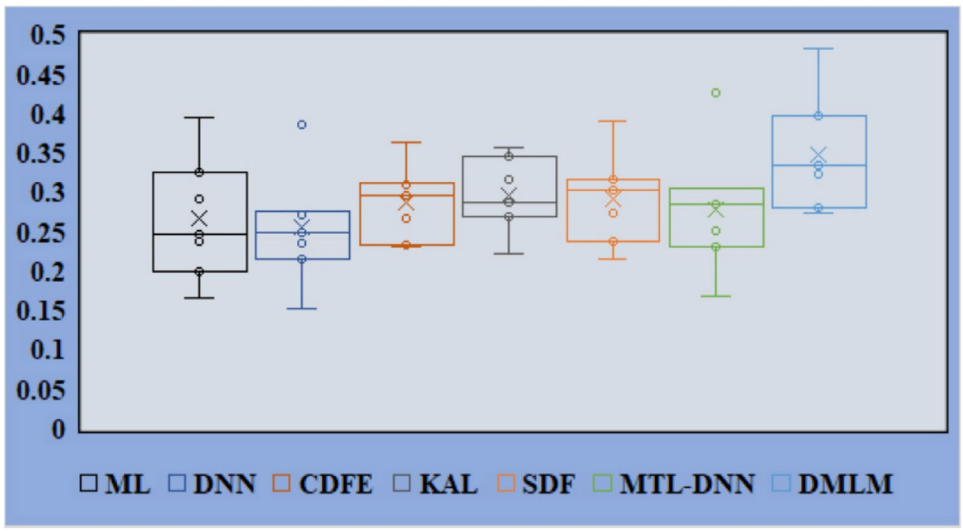

Fig. 13. Comparison of PofB20 scores for ML, DNN, CDFE, KAL, SDF, MTL-DNN, and DMLM under cross projects.

Cliff's delta	Effective levels	Cliff's delta	Effective levels
$0.474 \leq \|\delta\|$	Large (L)	$0.147 \leq \|\delta\| < 0.33$	Small (S)
$0.33 \leq \|\delta\| < 0.474$	Medium (M)	$\|\delta\| < 0.147$	Negligible (N)

Table 14. Correlation between the absolute values of Cliff's delta ($|\delta|$) and their respective effectiveness levels.

Projects	DMLM vs ML	DMLM vs DNN	DMLM vs CDFE	DMLM vs KAL	DMLM vs SDF	DMLM vs MTL-DNN
TV	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Messaging	+L(<0.05)	+L(<0.05)	(-N)	-L(<0.05)	+L(<0.05)	+L(<0.05)
Contacts	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	(+N)
Gallery	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Documents UI	+L(<0.05)	(+S)	(+S)	(+N)	-L(<0.05)	+L(<0.05)
Bluetooth	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	-L(<0.05)
Browser	+L(<0.05)	+L(<0.05)	+L(<0.05)	(+S)	+L(<0.05)	+L(<0.05)
Camera	+L(<0.05)	-L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Email	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Win/Tie/Loss	9/0/0	7/1/1	7/2/0	6/2/1	8/0/1	7/1/1

Table 15. WIN/TIE/LOSS indicator on F1 scores of DMLM and other baseline methods.

Projects	DMLM vs ML	DMLM vs DNN	DMLM vs CDFE	DMLM vs KAL	DMLM vs SDF	DMLM vs MTL-DNN
TV	(+S)	+L(<0.05)	+L(<0.05)	-L(<0.05)	+L(<0.05)	-L(<0.05)
Messaging	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Contacts	+L(<0.05)	(+M)	+L(<0.05)	-L(<0.05)	+L(<0.05)	+L(<0.05)
Gallery	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Documents UI	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Bluetooth	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Browser	-L(<0.05)	(-N)	+L(<0.05)	(+S)	+L(<0.05)	+L(<0.05)
Camera	(+M)	-L(<0.05)	+L(<0.05)	(+N)	+L(<0.05)	+L(<0.05)
Email	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Win/Tie/Loss	6/2/1	6/2/1	9/0/0	5/2/2	9/0/0	8/0/1

Table 16. WIN/TIE/LOSS indicator on AUC scores of DMLM and other baseline methods.

The WSRT is a non-parametric test that does not assume a specific data distribution and is commonly used to compare two paired samples. If the resulting p-value is less than 0.05, it indicates a statistically significant difference; otherwise, it is considered insignificant.

To mitigate potential biases from multiple comparisons in this study, we incorporate the Win/Tie/Loss metric to assess model performance comprehensively. Furthermore, Cliff's delta ($|\delta|$) is utilized to measure the practical significance of the observed differences between datasets, offering more profound insights into the comparative effectiveness of the models. The interpretation of absolute Cliff's delta values about practical significance levels is summarized in Table 14.

Table 15 presents the Win/Tie/Loss analysis results based on F1 scores, providing a comparative evaluation of the models. Each column in the table includes the p-values obtained from the Wilcoxon Signed Rank Test (WSRT) and the corresponding Cliff's delta values. The original value is retained if the WSRT p-value is greater than or equal to 0.05. Conversely, if the p-value falls below 0.05, it is represented as "<" to indicate statistical significance. Additionally, the practical importance of Cliff's delta values is assessed based on the predefined thresholds in Table 14. The sign of the delta value ("+" or "−") is used to denote whether the observed effect favors the proposed model or the baseline methods, providing further insights into the relative strengths of each approach. For example, when comparing the DMLM method with the ML approach on the "TV" project, the Cliff's delta value is greater than 0.474, and the p-value is less than 0.05. Consequently, the entry for "DMLM vs. ML" is recorded as "$+L(< 0.05)$". Following the Win/Tie/Loss measure criteria, the DMLM model is classified as a "Win."

Additionally, Table 16 presents the Win/Tie/Loss analysis results based on AUC values, while Table 17 provides the corresponding evaluation for the MCC metric, and Table 18 reports the results for POFB20. A comprehensive examination of the WSRT p-values, Cliff's delta values, and the "Win/Tie/Loss" outcomes across Tables 15 to 18 reveals a consistent trend– the proposed DMLM model demonstrates superior performance compared to the baseline approaches across multiple evaluation metrics. These results further validate the

Projects	DMLM vs ML	DMLM vs DNN	DMLM vs CDFE	DMLM vs KAL	DMLM vs SDF	DMLM vs MTL-DNN
TV	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Messaging	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	-L(<0.05)	+L(<0.05)
Contacts	+L(<0.05)	-L(<0.05)	+L(<0.05)	(+S)	+L(<0.05)	+L(<0.05)
Gallery	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Documents UI	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Bluetooth	+L(<0.05)	(+N)	+L(<0.05)	-L(<0.05)	+L(<0.05)	+L(<0.05)
Browser	-L(<0.05)	(+S)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Camera	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Email	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	(+S)
Win/Tie/Loss	8/0/1	6/2/1	9/0/0	7/1/1	8/0/1	8/1/0

Table 17. WIN/TIE/LOSS indicator on MCC scores of DMLM and other baseline methods.

Projects	DMLM vs ML	DMLM vs DNN	DMLM vs CDFE	DMLM vs KAL	DMLM vs SDF	DMLM vs MTL-DNN
TV	+L(<0.05)	+L(<0.05)	-L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Messaging	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Contacts	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	(+S)
Gallery	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	(+S)
Documents UI	+L(<0.05)	+L(<0.05)	+L(<0.05)	(+S)	+L(<0.05)	+L(<0.05)
Bluetooth	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	-L(<0.05)	+L(<0.05)
Browser	+L(<0.05)	+L(<0.05)	+L(<0.05)	(-S)	(+N)	-L(<0.05)
Camera	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Email	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)	+L(<0.05)
Win/Tie/Loss	9/0/0	9/0/0	8/0/1	7/2/0	7/1/1	6/2/1

Table 18. WIN/TIE/LOSS indicator on POFB20 scores of DMLM and other baseline methods.

robustness and effectiveness of the DMLM model in handling diverse predictive tasks. This holds in both effort-aware and non-effort-aware conditions, further validating the superior performance of DMLM across multiple evaluation metrics.

Threats to validity

Datasets selection

To validate our experiments, we tested a subset of nine real-world Android applications sourced from Git Android repositories. However, it is essential to acknowledge that these selected projects may not fully represent the entire spectrum of mobile apps. Additionally, our evaluation was solely focused on Android applications, meaning that the applicability of our model to other types of mobile apps, such as those developed for Apple's iOS platform, has not been assessed. Consequently, the results produced by our proposed method may differ when applied to apps outside of the nine selected Android projects or to those developed for other platforms like iOS.

Labeling data

Following previous studies[7,31], data samples are automatically labeled as defective or clean using the annotate or blame functions in the version control system (VCS). This labeling process can introduce noise, as acknowledged in prior work. Noisy data can degrade the performance of defect prediction models. To address this, we applied the data filtering algorithms described in Section 5.3 to mitigate the influence of noisy labels. These filtering techniques help remove outlier data points that could negatively impact model training. Taking steps to reduce label noise and filter anomalous data samples will help safeguard our models against potential harm from inaccuracies in the automated labeling process.

Implementation of baselines

We evaluate the performance of our prediction model through a comparative analysis, where it is tested against six leading, state-of-the-art methods: ML, DNN, CDFE, KAL, SDF, and MTL-DNN. Due to the unavailability of the original versions of these methods, we conducted our implementations. While we adhere to the instructions provided in their respective publications, some implementation intricacies from the original versions may have been inadvertently omitted in our re-implementations. To evaluate the performance of our newly implemented methods, we conducted tests using the same datasets employed in the original works. We are reasonably confident that our re-implementations faithfully capture the essence of these methods and accurately reflect their performance characteristics.

Conclusion

In this study, we introduced a novel approach, DMLM, designed to predict mobile application defects. Our method uses VCS and leverages the 'Javalang' and 're' libraries to extract code-based and process-related metrics from the current version of the source code and its previous releases. We employed a Deep CNN to develop a robust predictive model that learns patterns from these extracted features. To evaluate the effectiveness of DMLM, we conducted comprehensive experiments on nine real-world Android applications sourced from Git Android repositories. Furthermore, we benchmarked its performance against six cutting-edge mobile App defect prediction techniques to ensure a thorough comparative analysis. Our results demonstrated that DMLM outperforms the existing methods under non-effort-aware cases regarding F1 scores, AUC, and MCC. DMLM also outperforms the existing methods under effort-aware cases regarding PofB20. Despite the proposed DMLM model being promising for mobile app defect prediction, it has some limitations. Firstly, the model was evaluated only on Android apps and may not be generalized to other platforms like iOS. Defect patterns could vary across different mobile ecosystems. Secondly, the model utilizes only code and process metrics, and it may not capture other factors that can affect the quality of mobile apps, such as sentence-level and comment-level feature learning mobile app defect prediction.

Future research can build on this work by extending the approach to multi-platform datasets and cross-project transfer learning, thereby improving its generalizability. Integrating semantic code embeddings such as CodeBERT or GraphCodeBERT could enrich the representation of structural and syntactic features beyond traditional metrics. Moreover, adopting multi-task learning to predict defect proneness and severity jointly would enhance the practical utility of the predictions. Finally, combining metric-based features with developer network metrics through social graph analysis presents a promising direction for capturing the human factors underlying defect occurrence.

Data availability

The datasets are available from the corresponding author upon reasonable request.

Received: 18 July 2025; Accepted: 29 September 2025
Published online: 04 November 2025

References

1. Shamsujjoha, M., Grundy, J., Li, L., Khalajzadeh, H. & Lu, Q. Developing mobile applications via model driven development: a systematic literature review. *Inf. Softw. Technol.* **140**, (2021).
2. He, Q. et al. Diversified third-party library prediction for mobile app development. *IEEE Transactions on Softw. Eng.* **48**, 150–165 (2020).
3. Albahri, O. et al. Selection of smartphone-based mobile applications for obesity management using an interval neutrosophic vague decision-making framework. *Eng. Appl. Artif. Intell.* **137**, (2024).
4. Muthusamy, R. & Charles, Y. R. High-precision malware detection in android apps using quantum explainable hierarchical interaction network. *Knowledge-Based Syst.* **310**, (2025).
5. Abdu, A. et al. Cross-project software defect prediction based on the reduction and hybridization of software metrics. *Alex. Eng. J.* **112**, 161–176 (2025).
6. Coelho, J. et al. Semantic similarity for mobile application recommendation under scarce user data. *Eng. Appl. Artif. Intell.* **121**, (2023).
7. Dong, F., Wang, J., Li, Q., Xu, G. & Zhang, S. Defect prediction in android binary executables using deep neural network. *Wirel. Pers. Commun.* **102**, 2261–2285 (2018).
8. Malhotra, R. An empirical framework for defect prediction using machine learning techniques with android software. *Appl. Soft Comput.* **49**, 1034–1050 (2016).
9. Ricky, M. Y., Purnomo, F. & Yulianto, B. Mobile application software defect prediction. In *2016 IEEE Symposium on Service-Oriented System Engineering (SOSE)*, 307–313 (IEEE, 2016).
10. Catolino, G., Di Nucci, D. & Ferrucci, F. Cross-project just-in-time bug prediction for mobile apps: An empirical assessment. In *2019 IEEE/ACM 6th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, 99–110 (IEEE, 2019).
11. Zhao, K., Xu, Z., Zhang, T., Tang, Y. & Yan, M. Simplified deep forest model based just-in-time defect prediction for android mobile apps. *IEEE Transactions on Reliab.* **70**, 848–859 (2021).
12. Cheng, T., Zhao, K., Sun, S., Mateen, M. & Wen, J. Effort-aware cross-project just-in-time defect prediction framework for mobile apps. *Front. Comput. Sci.* **16**, (2022).
13. Huang, Q., Li, Z. & Gu, Q. Multi-task deep neural networks for just-in-time software defect prediction on mobile apps. *Concurr. Comput. Pract. Exp.* 7664 (2023).
14. van Dinter, R., Catal, C., Giray, G. & Tekinerdogan, B. Just-in-time defect prediction for mobile applications: using shallow or deep learning? *Softw. Qual. J.* 1–22 (2023).
15. Xu, Z. et al. Effort-aware just-in-time bug prediction for mobile apps via cross-triplet deep feature embedding. *IEEE Transactions on Reliab.* **71**, 204–220 (2021).
16. Abdu, A., Zhai, Z., Abdo, H. A. & Algabri, R. Software defect prediction based on deep representation learning of source code from contextual syntax and semantic graph. *IEEE Transactions on Reliab.* (2024).
17. Wu, W., Wang, S., Liu, B., Shao, Y. & Xie, W. A novel software defect prediction approach via weighted classification based on association rule mining. *Eng. Appl. Artif. Intell.* **129**, (2024).
18. Zhou, X., Han, D. & Lo, D. Bridging expert knowledge with deep learning techniques for just-in-time defect prediction. *Empir. Softw. Eng.* **30**, 1–44 (2025).
19. Tang, Y., Zhou, Y., Yang, C., Du, Y. & Yang, M.-S. Instance gravity oversampling method for software defect prediction. *Inf. Softw. Technol.* **179**, (2025).
20. Jiang, S., He, Z., Chen, Y., Zhang, M. & Ma, L. Mobile application online cross-project just-in-time software defect prediction framework. *ACM Transactions on Softw. Eng. Methodol.* (2024).
21. Feng, S. et al. Coste: Complexity-based oversampling technique to alleviate the class imbalance problem in software defect prediction. *Inf. Softw. Technol.* **129**, (2021).
22. Abdu, A. et al. Semantic and traditional feature fusion for software defect prediction using hybrid deep learning model. *Sci. Reports* **14**, 14771 (2024).

23. Dar, A. W. & Farooq, S. U. Handling class overlap and imbalance using overlap driven under-sampling with balanced random forest in software defect prediction. *Innov. Syst. Softw. Eng.* 1–21 (2024).
24. Bennin, K. E., Keung, J., Phannachitta, P., Monden, A. & Mensah, S. Mahakil: Diversity based oversampling approach to alleviate the class imbalance issue in software defect prediction. *IEEE Transactions on Softw. Eng.* **44**, 534–550 (2017).
25. Arun, C. & Lakshmi, C. Genetic algorithm-based oversampling approach to prune the class imbalance issue in software defect prediction. *Soft Comput.* **26**, 12915–12931 (2022).
26. Yedida, R. & Menzies, T. On the value of oversampling for deep learning in software defect prediction. *IEEE Transactions on Softw. Eng.* **48**, 3103–3116 (2021).
27. Goyal, S. Handling class-imbalance with knn (neighbourhood) under-sampling for software defect prediction. *Artif. Intell. Rev.* **55**, 2023–2064 (2022).
28. Tantithamthavorn, C., Hassan, A. E. & Matsumoto, K. The impact of class rebalancing techniques on the performance and interpretation of defect prediction models. *IEEE Transactions on Softw. Eng.* **46**, 1200–1219 (2018).
29. Carka, J., Esposito, M. & Falessi, D. On effort-aware metrics for defect prediction. *Empir. Softw. Eng.* **27**, 152 (2022).
30. Liu, C., Yang, D., Xia, X., Yan, M. & Zhang, X. A two-phase transfer learning model for cross-project defect prediction. *Inf. Softw. Technol.* **107**, 125–136 (2019).
31. Wang, S., Liu, T., Nam, J. & Tan, L. Deep semantic feature learning for software defect prediction. *IEEE Transactions on Softw. Eng.* **46**, 1267–1293 (2018).

Author contributions

Conceptualization, A.A., H.A.A., I.U., J.K., Y.H.G., R.A.; methodology, A.A., H.A.A., I.U., J.K., Y.H.G., R.A.; software, A.A., H.A.A.; validation, A.A., H.A.A., I.U., J.K., Y.H.G., R.A.; formal analysis, A.A., H.A.A., I.U., J.K., Y.H.G., R.A.; investigation, A.A., I.U., J.K.; resources, A.A., H.A.A, J.K.; data curation, A.A., H.A.A.; writing-original draft preparation, A.A., H.A.A.; writing-review and editing, I.U., J.K., Y.H.G., R.A.; visualization, A.A., H.A.A.; supervision, Y.H.G., R.A.; project administration, Y.H.G., R.A.; funding acquisition, Y.H.G., R.A. All authors have read and agreed to the published version of the manuscript. All authors discussed the results and commented on the manuscript.

Funding

This research was supported by the IITP (Institute of Information & Communications Technology Planning & Evaluation)-ITRC (Information Technology Research Center) grant funded by the Korea government (Ministry of Science and ICT) (IITP-2025-RS-2024-00437191) and by Sejong University Industry-University Cooperation Foundation under the project number (2025-0247-01).

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to Y.H.G. or R.A.